

PORTADA

Análisis y Diseño de Sistemas II

Semana 16 — Laboratorio: Pruebas Unitarias sobre el Proyecto

Universidad de Antioquía · Ingeniería de Sistemas



APERTURA

**Hoy la teoría se vuelve código
que corre sobre su propio proyecto**

TRANSICIÓN

De TDD en teoría, a pruebas reales en su código

El martes se vieron los fundamentos y frameworks de pruebas unitarias; el miércoles, el ciclo Red-Green-Refactor de TDD y las buenas prácticas para que una prueba realmente aporte. Hoy, en vez de un ejemplo de clase, cada equipo escribe pruebas unitarias **sobre su propio proyecto** de Fábrica Escuela.

Esta sesión es también el **cierre de la Unidad 6 — Calidad en el Diseño de Software** completa: deuda técnica, código limpio y pruebas unitarias, las tres semanas que empezaron el lunes pasado.

SÍNTESIS

Las tres piernas de la calidad del diseño

Sesión	Tema	Qué aporta
Semana 15 · martes	Deuda técnica	Nombrar y hacer visible el costo de los atajos de diseño.
Semana 15 · miércoles y viernes	Código limpio	Nombres, funciones, comentarios, refactorización y code smells: cómo pagar esa deuda.
Semana 16 · martes y miércoles	Pruebas unitarias	La red de seguridad que hace posible refactorizar sin miedo, y TDD como guía de diseño.

Ninguna de las tres funciona bien de forma aislada: código limpio sin pruebas es frágil de mantener; pruebas sobre código sucio son difíciles de escribir; y deuda técnica sin visibilidad nunca se paga.

INSTRUCCIONES

Laboratorio de cierre de la Unidad 6

Pruebas Unitarias sobre el Proyecto — 90 minutos, en la sala de Zoom asignada a cada equipo.

- **Trabajo en equipo** — el mismo equipo del proyecto de Fábrica Escuela conformado desde la semana 2.
- **Herramienta:** el framework de pruebas ya elegido por cada equipo según su stack tecnológico (JUnit 5, xUnit/NUnit, pytest, Jest, u otro equivalente).
- Cada equipo debe escribir pruebas unitarias para **al menos 3 métodos o funciones clave** de su propio proyecto, aplicando **TDD** (Red-Green-Refactor) o, como mínimo, la estructura **AAA** vista en las dos sesiones anteriores.



INSTRUCCIONES

Cómo elegir los 3 métodos o funciones

- Prioricen **lógica de negocio real** del dominio de su proyecto (cálculos, validaciones, reglas) por encima de código trivial (getters/setters) — ahí es donde una prueba unitaria aporta más valor.
- Si identificaron deuda técnica o code smells en las semanas anteriores en alguno de esos métodos, es un buen candidato: escribir la prueba primero (TDD) ayuda a confirmar el comportamiento actual antes de refactorizar con seguridad.
- Para cada método, sigan el ciclo visto el miércoles: escriban primero la prueba que falla (**Red**), el código mínimo que la hace pasar (**Green**), y si alcanza el tiempo, una mejora estructural (**Refactor**) sin romper la prueba.

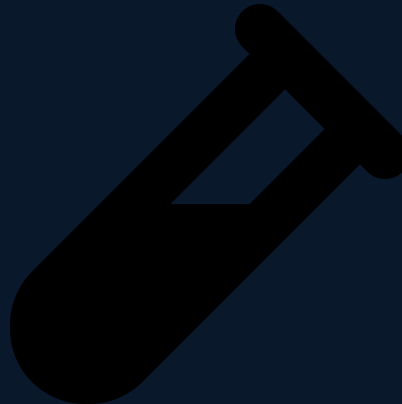
Recuerden: nombres descriptivos tipo `debe_devolver_error_cuando_el_monto_es_negativo` , una aserción lógica por prueba.

INSTRUCCIONES

Entregable y cierre del laboratorio

- **Formato exacto:** el código de las pruebas unitarias, escrito con el framework del stack del equipo, ejecutándose correctamente contra el código real del proyecto.
- Deben ser pruebas de **al menos 3 métodos o funciones clave**, siguiendo AAA (o el ciclo TDD completo) y las buenas prácticas de nombres y aserciones vistas el miércoles.
- **Subir el código de las pruebas al repositorio del equipo** antes de que termine la sesión — en la misma rama o carpeta de pruebas que ya use el proyecto.

La entrega cierra al terminar los 90 minutos.



SÍNTESIS

Qué debieron lograr en este laboratorio

1. Escribir pruebas unitarias reales para al menos 3 métodos o funciones clave de su propio proyecto.
2. Aplicar TDD (Red-Green-Refactor) o, como mínimo, la estructura AAA en cada prueba.
3. Usar nombres descriptivos y una aserción lógica por prueba, evitando pruebas frágiles acopladas a la implementación.
4. Dejar el código de las pruebas subido al repositorio del equipo, ejecutándose correctamente.

SÍNTESIS · CIERRE DE LA UNIDAD 6

Deuda técnica, código limpio y pruebas: las tres piernas de la calidad

Con este laboratorio se cierra la **Unidad 6 — Calidad en el Diseño de Software** completa. En dos semanas se cubrió el concepto de deuda técnica y su origen en Ward Cunningham; código limpio según Robert C. Martin —nombres, funciones, comentarios, refactorización y el catálogo de code smells de Martin Fowler—; y pruebas unitarias, desde sus fundamentos hasta TDD de Kent Beck.

Las tres ideas se sostienen entre sí: **nombrar la deuda** la hace visible, **código limpio** reduce su interés, y **pruebas unitarias** hacen seguro pagarla — hoy, cada equipo aplicó las tres directamente sobre su propio proyecto de Fábrica Escuela.



CIERRE

Cierre de la Unidad 6

Con esto se cierra la Unidad 6. El curso entra ahora en su recta final: la **siguiente unidad es el Cierre de Curso**, con la integración final de todo el diseño construido a lo largo del semestre — desde el modelo de dominio de la Unidad 2, pasando por la arquitectura, SOLID, GRASP, GoF y antipatrones, hasta la calidad de diseño vista en estas dos últimas semanas.

